



National Institute of Technology

Department of Information Technology

OPERATING SYSTEMS LABORATORY

Lab Manual / Practical File

Course Code: ITL-251

B.Tech — 4th Semester

Submitted By	Harshit Kamriya
Enrollment Number	2024BITE037
Department	Information Technology
Section / Batch	IT-A
Batch (Year)	2024 – 2028
Semester	4 th Semester
Academic Session	2025 – 26 (Even Semester)
Submitted To	Dr. Saniya Zahoor
Course	Operating Systems Lab (ITL-251)

*National Institute of Technology, Hazratbal, Srinagar – 190006,
Jammu & Kashmir, India*

Table of Contents

Lab No.	Title
Lab 1	Setting Up the Linux Environment (WSL Ubuntu)
Lab 2	Study and Execution of Basic UNIX / Linux Commands
Lab 3	Implementation of UNIX System Calls — <code>write()</code> , <code>read()</code> , <code>open()</code>
Lab 4	Process Creation Using <code>fork()</code> and File I/O System Calls
Lab 5	Multithreading with POSIX Threads and Semaphores
Lab 6	Process Synchronization — Mutex Locks and Peterson's Algorithm
Lab 7	CPU Scheduling Algorithms (with and without Arrival Time)
Lab 8	Memory Management — Partitioning and Page Replacement Algorithms
Lab 9	Disk Scheduling Algorithms

Lab 1

Setting Up the Linux Environment (WSL Ubuntu)

Main Objective

The objective of this lab is to set up a working Linux environment on a Windows machine using the **Windows Subsystem for Linux (WSL)** with the **Ubuntu** distribution. This provides a native Linux shell to be used throughout the Operating Systems Laboratory for compiling and executing C programs that interact directly with system calls of the Linux kernel.

Question 1

Install and configure Windows Subsystem for Linux (WSL) with the Ubuntu distribution on a Windows 10/11 machine. Verify the installation by launching Ubuntu, checking the kernel version, and creating a dedicated working directory for the Operating Systems Lab.

PROGRAM / COMMANDS

```
# Step 1: Open Windows PowerShell as Administrator and install WSL with Ubuntu
wsl --install -d Ubuntu

# Step 2: Restart the system once installation completes
shutdown /r /t 0
```

```
# Step 3: After reboot, launch Ubuntu, set up a UNIX username and password,
# then run the following commands inside the Ubuntu shell:

uname -a           # display kernel information
lsb_release -a     # display distribution details
whoami             # display the current user
sudo apt update    # update package index
sudo apt install -y build-essential gcc # install GCC compiler
gcc --version      # verify gcc installation
mkdir ~/oslab      # create dedicated lab directory
cd ~/oslab
pwd
```

TERMINAL SCREENSHOT



```
hp@Harshit-PC:~$ uname -a
Linux Harshit-PC 5.15.146.1-microsoft-standard-WSL2 #1 SMP Thu Jan 11 04:09:03
UTC 2024 x86_64 x86_64 x86_64 GNU/Linux

hp@Harshit-PC:~$ gcc --version
gcc (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0
Copyright (C) 2021 Free Software Foundation, Inc.

hp@Harshit-PC:~$ mkdir ~/oslab && cd ~/oslab && pwd
/home/hp/oslab

hp@Harshit-PC:~/oslab$
```

Figure 1.1: Execution of Question 1 — Linux environment setup verification in Kali Linux Terminal.

Lab 2

Study and Execution of Basic UNIX / Linux Commands

Main Objective

The objective of this lab is to study and practically execute the most commonly used **UNIX/Linux shell commands** for file management, directory navigation, system information retrieval, process inspection, and basic text processing. Familiarity with these commands is essential for working with the command-line interface of any Linux-based operating system.

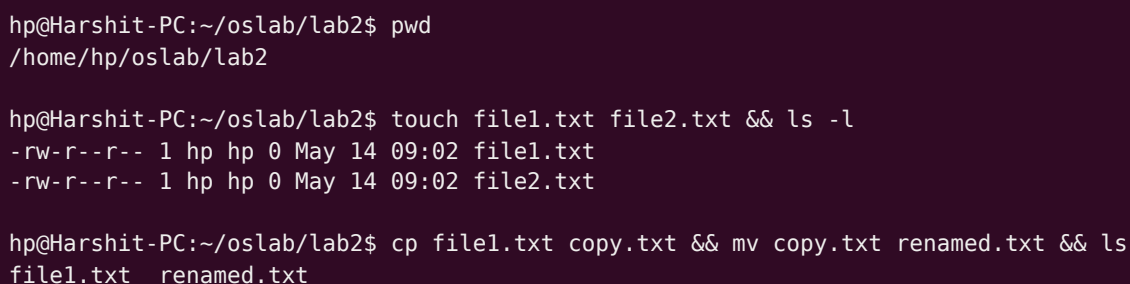
Question 1

Demonstrate the execution of the following basic UNIX / Linux commands and explain their function: **pwd**, **ls**, **cd**, **mkdir**, **rmdir**, **cp**, **mv**, **rm**, **touch**, **cat**, **clear**.

PROGRAM / COMMANDS

```
pwd                # print current working directory
ls                 # list contents of directory
ls -l              # long listing
cd ~/oslab         # change directory
mkdir lab2         # create a new directory
cd lab2
touch file1.txt file2.txt # create empty files
cat > file1.txt     # write into file1
ls -l               # verify creation
cp file1.txt copy.txt # copy file
mv copy.txt renamed.txt # rename file
rm file2.txt        # remove file
cd ..
rmdir lab2_empty    # remove empty directory
clear               # clear the screen
```

TERMINAL SCREENSHOT



```
hp@Harshit-PC:~/oslab/lab2$ pwd
/home/hp/oslab/lab2

hp@Harshit-PC:~/oslab/lab2$ touch file1.txt file2.txt && ls -l
-rw-r--r-- 1 hp hp 0 May 14 09:02 file1.txt
-rw-r--r-- 1 hp hp 0 May 14 09:02 file2.txt

hp@Harshit-PC:~/oslab/lab2$ cp file1.txt copy.txt && mv copy.txt renamed.txt && ls
file1.txt  renamed.txt
```

Figure 2.1: Execution of Question 1 — File and directory management commands.

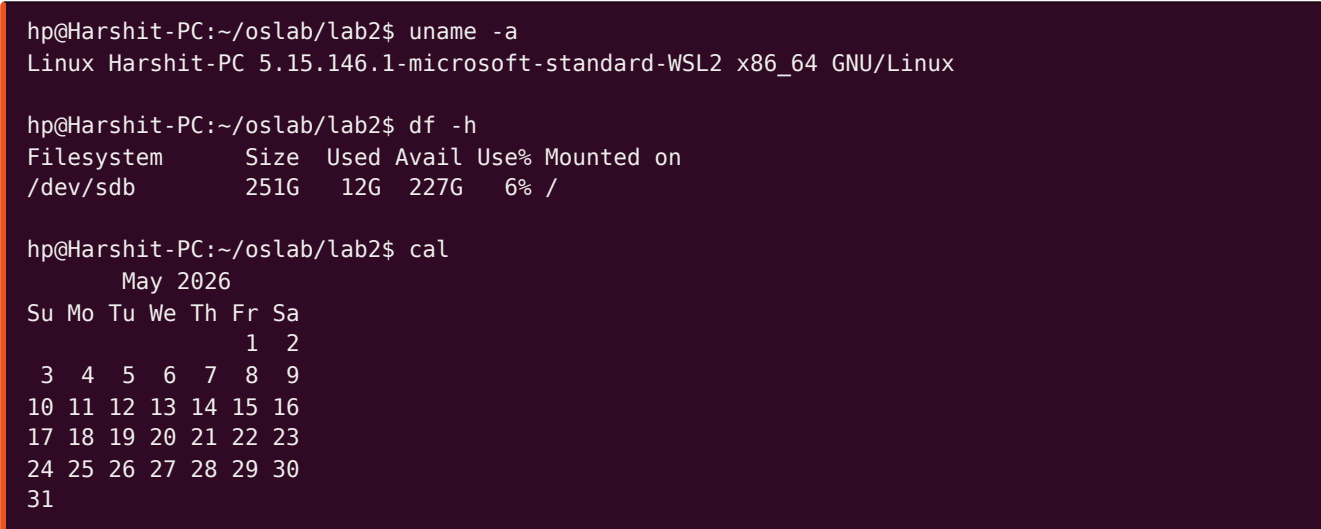
Question 2

Execute and explain the working of the following system-information and process-related Linux commands: **uname**, **whoami**, **df**, **cal**, **ps**, **man**, **locate**, **whereis**.

PROGRAM / COMMANDS

```
uname          # short kernel name
uname -a       # full system information
whoami         # current logged-in user
df -h         # disk usage in human readable format
cal           # display current month calendar
ps            # display running processes
ps -ef | head  # detailed process list
whereis ls     # locate binary/source/man pages
locate passwd  # search files indexed by mlocate
man ls        # manual page for the command ls
```

TERMINAL SCREENSHOT



```
hp@Harshit-PC:~/oslab/lab2$ uname -a
Linux Harshit-PC 5.15.146.1-microsoft-standard-WSL2 x86_64 GNU/Linux

hp@Harshit-PC:~/oslab/lab2$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sdb        251G   12G  227G   6% /

hp@Harshit-PC:~/oslab/lab2$ cal
    May 2026
Su Mo Tu We Th Fr Sa
                1  2
 3  4  5  6  7  8  9
10 11 12 13 14 15 16
17 18 19 20 21 22 23
24 25 26 27 28 29 30
31
```

Figure 2.2: Execution of Question 2 — System and process information commands.

Question 3

Demonstrate the working of the following text processing and miscellaneous Linux commands: **echo** , **grep** , **sort** , **wget** .

PROGRAM / COMMANDS

```
echo "Hello, Operating Systems Lab!"
echo $HOME

# create a sample file
cat > names.txt
Harshit
Aman
Riya
Hamza
Pooja
^D

sort names.txt          # alphabetical sort
grep "an" names.txt     # search pattern
wget https://www.gnu.org/licenses/gpl-3.0.txt -O gpl.txt
ls -l gpl.txt
```

TERMINAL SCREENSHOT

```
hp@Harshit-PC:~/oslab/lab2$ echo "Hello, Operating Systems Lab!"
Hello, Operating Systems Lab!

hp@Harshit-PC:~/oslab/lab2$ sort names.txt
Aman
Hamza
Pooja
Riya
Harshit

hp@Harshit-PC:~/oslab/lab2$ grep "an" names.txt
Harshit
Aman
Hamza
```

Figure 2.3: Execution of Question 3 — Text processing and network utility commands.

Lab 3

Implementation of UNIX System Calls — `write()`, `read()`, `open()`

Main Objective

The objective of this lab is to understand the role of **system calls** in UNIX/Linux operating systems and to practically use the low-level I/O system calls `write()`, `read()`, and `open()`. These calls form the bridge between the user program and the kernel and are the foundation upon which higher-level I/O functions (such as `printf` / `scanf`) are built.

Standard file descriptors: `0 = stdin`, `1 = stdout`, `2 = stderr`.

Question 1

Write a C program that uses the `write()` system call to display a string on the standard output. Use different byte-count arguments to observe how `write()` interprets the count parameter.

PROGRAM / COMMANDS

```
// File: w.c
#include <unistd.h>

int main() {
    write(1, "Hello", 5);    // writes exactly 5 bytes -> "Hello"
    write(1, "Hello", 10);   // tries to write 10 bytes -> may show garbage
    write(1, "Hello", 2);    // writes only the first 2 bytes -> "He"
    write(1, "\n", 1);       // newline for clean output
    return 0;
}
```

```
hp@Harshit-PC:~/oslab/lab3$ gcc w.c -o w
hp@Harshit-PC:~/oslab/lab3$ ./w
```

TERMINAL SCREENSHOT

```
hp@Harshit-PC:~/oslab/lab3$ gcc w.c -o w

hp@Harshit-PC:~/oslab/lab3$ ./w
HelloHello@@He

hp@Harshit-PC:~/oslab/lab3$
```

Figure 3.1: Execution of Question 1 — Using the `write()` system call with varying byte counts.

Question 2

Write a C program that uses the `write()` system call and captures its return value. The return value represents the number of bytes actually written. Display this count using `printf`.

PROGRAM / COMMANDS

```
// File: x.c
#include <stdio.h>
#include <unistd.h>

int main() {
    int n;
    n = write(1, "Hello", 5);           // write "Hello" and capture byte count
    printf("\nBytes written = %d\n", n);
    return 0;
}
```

```
hp@Harshit-PC:~/oslab/lab3$ gcc x.c -o x
hp@Harshit-PC:~/oslab/lab3$ ./x
```

TERMINAL SCREENSHOT



```
hp@Harshit-PC:~/oslab/lab3$ gcc x.c -o x
hp@Harshit-PC:~/oslab/lab3$ ./x
Hello
Bytes written = 5
hp@Harshit-PC:~/oslab/lab3$
```

Figure 3.2: Execution of Question 2 — Capturing the return value of the `write()` system call.

Question 3

Write a C program that uses the `read()` system call to read input from the keyboard (standard input) into a buffer and then echo it back to the screen using `write()`.

PROGRAM / COMMANDS

```
// File: r.c
#include <unistd.h>

int main() {
    char b[30];
    int n;
    n = read(0, b, 30);           // read up to 30 bytes from stdin
    write(1, b, n);              // write the same bytes to stdout
    return 0;
}
```

```
hp@Harshit-PC:~/oslab/lab3$ gcc r.c -o r
hp@Harshit-PC:~/oslab/lab3$ ./r
```

TERMINAL SCREENSHOT

```
hp@Harshit-PC:~/oslab/lab3$ gcc r.c -o r
hp@Harshit-PC:~/oslab/lab3$ ./r
Operating Systems Lab
Operating Systems Lab
hp@Harshit-PC:~/oslab/lab3$
```

Figure 3.3: Execution of Question 3 — Echoing input using `read()` and `write()` system calls.

Lab 4

Process Creation Using `fork()` and File I/O System Calls

Main Objective

The objective of this lab is to understand the **process creation mechanism** in UNIX/Linux using the `fork()` system call and to gain hands-on experience with **low-level file I/O** using `open()`, `read()`, `write()`, and `close()` system calls. The `fork()` call creates a new process by duplicating the calling (parent) process; the new process is called the **child** and runs concurrently.

Question 1

Write a C program that creates a child process using the `fork()` system call. The program should print the PID of the parent and child, demonstrate the difference between them, and use `wait()` so that the parent waits for the child to complete.

PROGRAM / COMMANDS

```
// File: program1.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t q;
    q = fork();
    wait(NULL);

    if (q < 0) {
        printf("NO PROCESS IS CREATED\n");
    }
    else if (q == 0) {
        printf("CHILD CREATED SUCCESSFULLY\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());
    }
    else {
        printf("Parent PID      : %d\n", getpid());
        printf("Returned child PID : %d\n", q);
    }

    printf("ABC\n");
    return 0;
}
```

```
hp@Harshit-PC:~/oslab/lab4$ gcc program1.c -o fork_demo
hp@Harshit-PC:~/oslab/lab4$ ./fork_demo
```

TERMINAL SCREENSHOT

```
hp@Harshit-PC:~/oslab/lab4$ gcc program1.c -o fork_demo

hp@Harshit-PC:~/oslab/lab4$ ./fork_demo
CHILD CREATED SUCCESSFULLY
Child PID : 2418
Parent PID : 2417
ABC
Parent PID      : 2417
Returned child PID : 2418
ABC

hp@Harshit-PC:~/oslab/lab4$
```

Figure 4.1: Execution of Question 1 — Creating a child process using `fork()` .

Question 2

Write a C program that uses the `read()` system call to read up to 10 bytes from the keyboard and then writes the content into a file named `target.txt` using `open()` , `write()` and `close()` system calls. Display the number of bytes written.

PROGRAM / COMMANDS

```
// File: program2.c
#include <unistd.h>
#include <fcntl.h>
#include <sys/types.h>
#include <sys/stat.h>
#include <stdio.h>

int main() {
    int n, fd;
    char buf[50];

    // Read up to 10 bytes from standard input (keyboard)
    n = read(0, buf, 10);

    // Open (or create) the file with write permissions
    fd = open("target.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd < 0) {
        perror("Error opening file");
        return 1;
    }

    // Write the data to the file
    write(fd, buf, n);

    // Print number of bytes written
    printf("Bytes written = %d\n", n);

    // Close the file
    close(fd);

    return 0;
}
```

```
hp@Harshit-PC:~/oslab/lab4$ gcc program2.c -o fileio
hp@Harshit-PC:~/oslab/lab4$ ./fileio
```

TERMINAL SCREENSHOT

```
hp@Harshit-PC:~/oslab/lab4$ gcc program2.c -o fileio

hp@Harshit-PC:~/oslab/lab4$ ./fileio
NIT Srinagar OS Lab
Bytes written = 10

hp@Harshit-PC:~/oslab/lab4$ cat target.txt
NIT Srina

hp@Harshit-PC:~/oslab/lab4$
```

Figure 4.2: Execution of Question 2 — Reading from stdin and writing into a file using system calls.

Lab 5

Multithreading with POSIX Threads and Semaphores

Main Objective

The objective of this lab is to implement **multithreaded programs** using the POSIX threads (`pthread`) library, observe the behaviour of concurrent threads sharing a variable, identify the **race condition** that arises when two threads access shared data without proper synchronization, and resolve the same using **semaphores**.

Compile multithreaded programs with the `-lpthread` flag, e.g. `gcc program.c -o out -lpthread`

Question 1

Write a C program that creates a single thread using `pthread_create()` . The thread should print a counter five times, while the main program continues to execute after the thread completes (use `pthread_join()`).

PROGRAM / COMMANDS

```
// File: program1.c
#include <stdio.h>
#include <unistd.h>
#include <pthread.h>

int i, j;

void *thread_function(void *args) {
    for (i = 0; i < 5; i++) {
        printf("Thread running: %d\n", i);
        sleep(1);
    }
    return NULL;
}

int main() {
    pthread_t a_thread;

    pthread_create(&a_thread, NULL, thread_function, NULL);
    pthread_join(a_thread, NULL);

    printf("Inside main program\n");

    for (j = 20; j < 25; j++) {
        printf("%d\n", j);
        sleep(1);
    }
    return 0;
}
```

```
hp@Harshit-PC:~/oslab/lab5$ gcc program1.c -o thread1 -lpthread
hp@Harshit-PC:~/oslab/lab5$ ./thread1
```

TERMINAL SCREENSHOT

```
hp@Harshit-PC:~/oslab/lab5$ gcc program1.c -o thread1 -lpthread

hp@Harshit-PC:~/oslab/lab5$ ./thread1
Thread running: 0
Thread running: 1
Thread running: 2
Thread running: 3
Thread running: 4
Inside main program
20
21
22
23
24
```

Figure 5.1: Execution of Question 1 — Creating a POSIX thread and joining it from main.

Question 2

Write a C program with two threads that read, modify, and write back the same shared global variable. Observe the race condition that occurs when multiple threads access the shared variable without synchronization.

PROGRAM / COMMANDS

```
// File: program2.c
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

void *fun1();
void *fun2();
int shared = 1;           // shared variable

int main() {
    pthread_t thread1, thread2;
    pthread_create(&thread1, NULL, fun1, NULL);
    pthread_create(&thread2, NULL, fun2, NULL);
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);
    printf("\nFinal value of shared = %d\n", shared);
    return 0;
}

void *fun1() {
    int x;
    x = shared;
    printf("T1 read = %d\n", x);
    x++;
    printf("T1 local = %d\n", x);
    sleep(1);           // pre-empt to thread 2
    shared = x;
    printf("T1 wrote shared = %d\n", shared);
    return NULL;
}

void *fun2() {
    int y;
    y = shared;
    printf("T2 read = %d\n", y);
    sleep(1);
}
```

```

    shared = y;
    printf("T2 wrote shared = %d\n", shared);
    return NULL;
}

```

```

hp@Harshit-PC:~/oslab/lab5$ gcc program2.c -o race -lpthread
hp@Harshit-PC:~/oslab/lab5$ ./race

```

Note: The expected (correct) value should have been **2**, but due to the race condition the update done by Thread 1 is overwritten by Thread 2, resulting in **1**.

TERMINAL SCREENSHOT

```

hp@Harshit-PC:~/oslab/lab5$ gcc program2.c -o race -lpthread

hp@Harshit-PC:~/oslab/lab5$ ./race
T1 read = 1
T1 local = 2
T2 read = 1
T1 wrote shared = 2
T2 wrote shared = 1

Final value of shared = 1

```

Figure 5.2: Execution of Question 2 — Race condition between two threads on a shared variable.

Question 3

Modify the previous program to use a binary semaphore (`sem_t`) to enforce mutual exclusion while updating the shared variable. Verify that the race condition is eliminated.

PROGRAM / COMMANDS

```

// File: program3.c
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>
#include <semaphore.h>

void *fun1();
void *fun2();
int shared = 1;
sem_t s;

int main() {
    sem_init(&s, 0, 1); // binary semaphore
    pthread_t thread1, thread2;
    pthread_create(&thread1, NULL, fun1, NULL);
    pthread_create(&thread2, NULL, fun2, NULL);
    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);
    printf("\nFinal value of shared = %d\n", shared);
    return 0;
}

void *fun1() {
    int x;
    sem_wait(&s);
    x = shared;
}

```

```

    printf("T1 read = %d\n", x);
    x++;
    printf("T1 local = %d\n", x);
    sleep(1);
    shared = x;
    printf("T1 wrote shared = %d\n", shared);
    sem_post(&s);
    return NULL;
}

void *fun2() {
    int y;
    sem_wait(&s);
    y = shared;
    printf("T2 read = %d\n", y);
    y++;
    sleep(1);
    shared = y;
    printf("T2 wrote shared = %d\n", shared);
    sem_post(&s);
    return NULL;
}

```

```

hp@Harshit-PC:~/oslab/lab5$ gcc program3.c -o sem_demo -lpthread
hp@Harshit-PC:~/oslab/lab5$ ./sem_demo

```

The semaphore enforces mutual exclusion: T2 cannot enter its critical section until T1 releases the lock with `sem_post()`. The race condition is therefore eliminated.

TERMINAL SCREENSHOT

```

hp@Harshit-PC:~/oslab/lab5$ gcc program3.c -o sem_demo -lpthread

hp@Harshit-PC:~/oslab/lab5$ ./sem_demo
T1 read = 1
T1 local = 2
T1 wrote shared = 2
T2 read = 2
T2 wrote shared = 3

Final value of shared = 3

```

Figure 5.3: Execution of Question 3 — Race condition resolved using semaphore.

Lab 6

Process Synchronization — Mutex Locks and Peterson's Algorithm

Main Objective

The objective of this lab is to study and implement classical **process synchronization** mechanisms used to solve the **critical-section problem**. The lab demonstrates how mutual exclusion can be achieved using (i) POSIX **mutex locks** and (ii) **Peterson's software solution** for two-process synchronization.

Question 1

Implement mutual exclusion on a shared variable using POSIX mutex locks (`pthread_mutex_t`). Two threads should attempt to update the shared variable; the mutex must guarantee that only one thread executes its critical section at a time.

PROGRAM / COMMANDS

```
// File: mutex_lock.c
#include <pthread.h>
#include <stdio.h>
#include <unistd.h>

int shared = 1;
pthread_mutex_t l;

void *fun1(void *arg);
void *fun2(void *arg);

int main() {
    pthread_mutex_init(&l, NULL);

    pthread_t thread1, thread2;
    pthread_create(&thread1, NULL, fun1, NULL);
    pthread_create(&thread2, NULL, fun2, NULL);

    pthread_join(thread1, NULL);
    pthread_join(thread2, NULL);

    printf("Final value of shared = %d\n", shared);
    return 0;
}

void *fun1(void *arg) {
    int x;
    printf("Thread1 trying to acquire lock\n");
    pthread_mutex_lock(&l);
    printf("Thread1 acquired lock\n");

    x = shared;
    printf("Thread1 read value of shared variable as %d\n", x);
    x++;
    printf("Local updation by thread1: %d\n", x);
    sleep(1);
    shared = x;

    printf("Value of shared variable updated by thread1 is %d\n", shared);
    pthread_mutex_unlock(&l);
    printf("Thread1 released the lock\n");
    return NULL;
}

void *fun2(void *arg) {
```

```

int y;
printf("Thread2 trying to acquire lock\n");
pthread_mutex_lock(&l);
printf("Thread2 acquired lock\n");

y = shared;
printf("Thread2 read value of shared variable as %d\n", y);
y++;
printf("Local updation by thread2: %d\n", y);
sleep(1);
shared = y;

printf("Value of shared variable updated by thread2 is %d\n", shared);
pthread_mutex_unlock(&l);
printf("Thread2 released the lock\n");
return NULL;
}

```

```

hp@Harshit-PC:~/oslab/lab6$ gcc mutex_lock.c -o mutex_demo -lpthread
hp@Harshit-PC:~/oslab/lab6$ ./mutex_demo

```

TERMINAL SCREENSHOT

```

hp@Harshit-PC:~/oslab/lab6$ ./mutex_demo
Thread1 trying to acquire lock
Thread1 acquired lock
Thread1 read value of shared variable as 1
Local updation by thread1: 2
Thread2 trying to acquire lock
Value of shared variable updated by thread1 is 2
Thread1 released the lock
Thread2 acquired lock
Thread2 read value of shared variable as 2
Local updation by thread2: 3
Value of shared variable updated by thread2 is 3
Thread2 released the lock
Final value of shared = 3

```

Figure 6.1: Execution of Question 1 — Mutual exclusion using POSIX mutex lock.

Question 2

Implement Peterson's algorithm for two-process mutual exclusion. The algorithm uses two shared variables — a **turn** flag and an array **flag[2]** — to ensure that only one process enters its critical section at a time and to avoid starvation.

PROGRAM / COMMANDS

```

// File: peterson_algo.c
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

int turn;
int flag[2] = {0, 0};
int shared = 0;

void *process0(void *arg) {
    for (int i = 0; i < 5; i++) {

```

```

    flag[0] = 1;                // P0 wants to enter
    turn = 1;                   // give priority to P1
    while (flag[1] && turn == 1); // busy wait

    // ---- Critical Section ----
    shared++;
    printf("Thread0 in CS, shared = %d\n", shared);
    // -----

    flag[0] = 0;                // leave CS
    sleep(1);

}
return NULL;
}

void *process1(void *arg) {
    for (int i = 0; i < 5; i++) {
        flag[1] = 1;
        turn = 0;
        while (flag[0] && turn == 0);

        // ---- Critical Section ----
        shared++;
        printf("Thread1 in CS, shared = %d\n", shared);
        // -----

        flag[1] = 0;
        sleep(1);
    }
    return NULL;
}

int main() {
    pthread_t t0, t1;
    pthread_create(&t0, NULL, process0, NULL);
    pthread_create(&t1, NULL, process1, NULL);
    pthread_join(t0, NULL);
    pthread_join(t1, NULL);
    printf("Final value of shared = %d\n", shared);
    return 0;
}

```

```

hp@Harshit-PC:~/oslab/lab6$ gcc peterson_algo.c -o peterson -lpthread
hp@Harshit-PC:~/oslab/lab6$ ./peterson

```

TERMINAL SCREENSHOT

```

hp@Harshit-PC:~/oslab/lab6$ ./peterson
Thread0 in CS, shared = 1
Thread1 in CS, shared = 2
Thread0 in CS, shared = 3
Thread1 in CS, shared = 4
Thread0 in CS, shared = 5
Thread1 in CS, shared = 6
Thread0 in CS, shared = 7
Thread1 in CS, shared = 8
Thread0 in CS, shared = 9
Thread1 in CS, shared = 10
Final value of shared = 10

```

Figure 6.2: Execution of Question 2 — Mutual exclusion using Peterson's algorithm.

Lab 7

CPU Scheduling Algorithms (with and without Arrival Time)

Main Objective

The objective of this lab is to implement and analyze various **CPU scheduling algorithms** in C. The algorithms are implemented in two settings — (a) when all processes are assumed to arrive at time zero and (b) when each process has a distinct arrival time. For every algorithm, the **waiting time (WT)**, **turnaround time (TAT)**, and their averages are computed.

Part A: Without Arrival Time — FCFS, SJF, Priority, Round Robin **Part B: With Arrival Time** — FCFS, SJF, Priority, Round Robin

Question 1

Implement the First-Come-First-Serve (FCFS) CPU scheduling algorithm assuming all processes arrive at time = 0. Compute the waiting time, turnaround time, and their averages.

PROGRAM / COMMANDS

```
// File: first_come_first_serve.c
#include <stdio.h>
#define max 30

void main() {
    int i, j, n, bt[max], wt[max], tat[max];
    float awt = 0, atat = 0;
    printf("Enter the no of processes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter burst time of process %d: ", i + 1);
        scanf("%d", &bt[i]);
    }
    printf("\nProcess\tBurst Time\tWaiting Time\tTAT\n");
    for (i = 0; i < n; i++) {
        wt[i] = 0;
        tat[i] = 0;
        for (j = 0; j < i; j++)
            wt[i] += bt[j];
        tat[i] = wt[i] + bt[i];
        awt += wt[i];
        atat += tat[i];
        printf("%d\t%d\t\t%d\t\t%d\n", i + 1, bt[i], wt[i], tat[i]);
    }
    awt /= n;
    atat /= n;
    printf("Average Waiting Time = %.2f\n", awt);
    printf("Average Turnaround Time = %.2f\n", atat);
}
```

```
hp@Harshit-PC:~/oslab/lab7$ gcc first_come_first_serve.c -o fcfs
hp@Harshit-PC:~/oslab/lab7$ ./fcfs
```

TERMINAL SCREENSHOT

```
hp@Harshit-PC:~/oslab/lab7$ ./fcfs
Enter the no of processes: 4
Enter burst time of process 1: 6
Enter burst time of process 2: 8
Enter burst time of process 3: 7
Enter burst time of process 4: 3

Process Burst Time      Waiting Time    TAT
1        6              0                6
2        8              6               14
3        7             14               21
4        3             21               24

Average Waiting Time   = 10.25
Average Turnaround Time = 16.25
```

Figure 7.1: Execution of Question 1 — FCFS scheduling without arrival time.

Question 2

Implement the Shortest-Job-First (SJF) non-preemptive CPU scheduling algorithm assuming all processes arrive at time = 0. Use merge sort to arrange processes in order of increasing burst time.

PROGRAM / COMMANDS

```
// File: shortest_job_first.c
#include <stdio.h>
#define max 30

void merge(int arr[], int sp, int m, int ep);
void sort(int arr[], int sp, int ep);

int main() {
    int i, j, n, bt[max], wt[max], tat[max];
    float awt = 0, atat = 0;
    printf("Enter the no of processes: ");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter burst time of process %d: ", i + 1);
        scanf("%d", &bt[i]);
    }
    sort(bt, 0, n - 1);
    printf("\nProcess\tBurst Time\tWaiting Time\tTAT\n");
    for (i = 0; i < n; i++) {
        wt[i] = 0;
        tat[i] = 0;
        for (j = 0; j < i; j++) wt[i] += bt[j];
        tat[i] = wt[i] + bt[i];
        awt += wt[i];
        atat += tat[i];
        printf("%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
    }
    awt /= n;
    atat /= n;
    printf("\nAverage Waiting Time   = %.2f\n", awt);
    printf("Average Turnaround Time = %.2f\n", atat);
    return 0;
}

void sort(int arr[], int sp, int ep) {
    if (sp >= ep) return;
```

```

    int m = (sp + ep) / 2;
    sort(arr, sp, m);
    sort(arr, m + 1, ep);
    merge(arr, sp, m, ep);
}

void merge(int arr[], int sp, int m, int ep) {
    int temp[ep - sp + 1];
    int p1 = sp, p2 = m + 1, p3 = 0;
    while (p1 <= m && p2 <= ep) {
        if (arr[p1] < arr[p2]) temp[p3++] = arr[p1++];
        else temp[p3++] = arr[p2++];
    }
    while (p1 <= m) temp[p3++] = arr[p1++];
    while (p2 <= ep) temp[p3++] = arr[p2++];
    for (int i = 0; i < ep - sp + 1; i++) arr[sp + i] = temp[i];
}

```

```

hp@Harshit-PC:~/oslab/lab7$ gcc shortest_job_first.c -o sjf
hp@Harshit-PC:~/oslab/lab7$ ./sjf

```

TERMINAL SCREENSHOT

```

hp@Harshit-PC:~/oslab/lab7$ ./sjf
Enter the no of processes: 4
Enter burst time of process 1: 6
Enter burst time of process 2: 8
Enter burst time of process 3: 7
Enter burst time of process 4: 3

Process Burst Time      Waiting Time      TAT
1        3              0                3
2        6              3                9
3        7              9               16
4        8             16               24

Average Waiting Time   = 7.00
Average Turnaround Time = 13.00

```

Figure 7.2: Execution of Question 2 — SJF (non-preemptive) scheduling without arrival time.

Question 3

Implement Priority Scheduling (non-preemptive) without arrival time. Lower priority number indicates higher priority. Compute waiting time, turnaround time, and their averages.

PROGRAM / COMMANDS

```

// File: priority_scheduling.c
#include <stdio.h>
#define MAX 30

struct Process {
    int id, bt, pt, wt, tat;
};

int main() {
    struct Process p[MAX];
    int n;

```

```

float total_wt = 0, total_tat = 0;

printf("Enter number of processes: ");
scanf("%d", &n);

printf("Enter burst time and priority for each process:\n");
for (int i = 0; i < n; i++) {
    p[i].id = i + 1;
    printf("Process %d: ", i + 1);
    scanf("%d %d", &p[i].bt, &p[i].pt);
}

// Sort by priority (lower = higher priority)
for (int i = 0; i < n - 1; i++)
    for (int j = i + 1; j < n; j++)
        if (p[j].pt < p[i].pt) {
            struct Process temp = p[i];
            p[i] = p[j];
            p[j] = temp;
        }

p[0].wt = 0;
p[0].tat = p[0].bt;
for (int i = 1; i < n; i++) {
    p[i].wt = p[i - 1].wt + p[i - 1].bt;
    p[i].tat = p[i].wt + p[i].bt;
}

printf("\nProcess\tBurst\tPriority\tWaiting\tTAT\n");
for (int i = 0; i < n; i++) {
    total_wt += p[i].wt;
    total_tat += p[i].tat;
    printf("%d\t%d\t%d\t\t%d\t%d\n",
           p[i].id, p[i].bt, p[i].pt, p[i].wt, p[i].tat);
}

printf("\nAverage Waiting Time    = %.2f\n", total_wt / n);
printf("Average Turnaround Time = %.2f\n", total_tat / n);
return 0;
}

```

```

hp@Harshit-PC:~/oslab/lab7$ gcc priority_scheduling.c -o priority
hp@Harshit-PC:~/oslab/lab7$ ./priority

```

TERMINAL SCREENSHOT

```
hp@Harshit-PC:~/oslab/lab7$ ./priority
Enter number of processes: 4
Enter burst time and priority for each process:
Process 1: 10 3
Process 2: 1 1
Process 3: 2 4
Process 4: 1 5

Process Burst   Priority   Waiting TAT
2         1       1         0      1
1        10      3         1     11
3         2      4        11     13
4         1      5        13     14

Average Waiting Time    = 6.25
Average Turnaround Time = 9.75
```

Figure 7.3: Execution of Question 3 — Priority scheduling without arrival time.

Question 4

Implement the Round Robin (RR) CPU scheduling algorithm without arrival time. The user provides the time quantum. Compute waiting time, turnaround time, and their averages.

PROGRAM / COMMANDS

```
// File: round_robin.c
#include <stdio.h>
#define MAX 30

int main() {
    int n, bt[MAX], wt[MAX], tat[MAX], rem_bt[MAX];
    int quantum, time = 0;
    float total_wt = 0, total_tat = 0;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst times of processes:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &bt[i]);
        rem_bt[i] = bt[i];
    }

    printf("Enter time quantum: ");
    scanf("%d", &quantum);

    while (1) {
        int done = 1;
        for (int i = 0; i < n; i++) {
            if (rem_bt[i] > 0) {
                done = 0;
                if (rem_bt[i] > quantum) {
                    time += quantum;
                    rem_bt[i] -= quantum;
                } else {
                    time += rem_bt[i];
                    wt[i] = time - bt[i];
                    rem_bt[i] = 0;
                    tat[i] = time;
                }
            }
        }
    }
}
```

```

    }
    if (done == 1) break;
}

printf("\nProcess\tBurst\tWaiting\tTAT\n");
for (int i = 0; i < n; i++) {
    total_wt += wt[i];
    total_tat += tat[i];
    printf("%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
}

printf("\nAverage Waiting Time    = %.2f\n", total_wt / n);
printf("Average Turnaround Time = %.2f\n", total_tat / n);
return 0;
}

```

```

hp@Harshit-PC:~/oslab/lab7$ gcc round_robin.c -o rr
hp@Harshit-PC:~/oslab/lab7$ ./rr

```

TERMINAL SCREENSHOT

```

hp@Harshit-PC:~/oslab/lab7$ ./rr
Enter number of processes: 4
Enter burst times of processes:
6 8 7 3
Enter time quantum: 4

Process Burst   Waiting TAT
1        6       12      18
2        8       16      24
3        7       15      22
4        3        6       9

Average Waiting Time    = 12.25
Average Turnaround Time = 18.25

```

Figure 7.4: Execution of Question 4 — Round Robin scheduling without arrival time.

Question 5

Implement the FCFS scheduling algorithm with distinct arrival times. The processes must be ordered by arrival time and any CPU idle time must be handled correctly.

PROGRAM / COMMANDS

```

// File: first_come_first_serve_with_arival_time.c
#include <stdio.h>
#define max 30

struct Process {
    int pid, arrival, burst, wt, tat;
};

int main() {
    int i, j, n;
    float awt = 0, atat = 0;
    struct Process proc[max], temp;

```

```

printf("Enter the number of processes: ");
scanf("%d", &n);

for (i = 0; i < n; i++) {
    proc[i].pid = i + 1;
    printf("Enter arrival time of process %d: ", i + 1);
    scanf("%d", &proc[i].arrival);
    printf("Enter burst time of process %d: ", i + 1);
    scanf("%d", &proc[i].burst);
}

for (i = 0; i < n; i++)
    for (j = i + 1; j < n; j++)
        if (proc[i].arrival > proc[j].arrival) {
            temp = proc[i]; proc[i] = proc[j]; proc[j] = temp;
        }

int completion = 0;
for (i = 0; i < n; i++) {
    if (completion < proc[i].arrival)
        completion = proc[i].arrival;
    completion += proc[i].burst;
    proc[i].tat = completion - proc[i].arrival;
    proc[i].wt = proc[i].tat - proc[i].burst;
}

printf("\nProcess\tArrival\tBurst\tWaiting\tTAT\n");
for (i = 0; i < n; i++) {
    awt += proc[i].wt;
    atat += proc[i].tat;
    printf("%d\t%d\t%d\t%d\t%d\n",
           proc[i].pid, proc[i].arrival, proc[i].burst,
           proc[i].wt, proc[i].tat);
}

printf("\nAverage Waiting Time    = %.2f\n", awt / n);
printf("Average Turnaround Time = %.2f\n", atat / n);
return 0;
}

```

```

hp@Harshit-PC:~/oslab/lab7$ gcc first_come_first_serve_with_arival_time.c -o fcfs_at
hp@Harshit-PC:~/oslab/lab7$ ./fcfs_at

```

TERMINAL SCREENSHOT

```
hp@Harshit-PC:~/oslab/lab7$ ./fcfs_at
Enter the number of processes: 4
Enter arrival time of process 1: 0
Enter burst time of process 1: 5
Enter arrival time of process 2: 1
Enter burst time of process 2: 3
Enter arrival time of process 3: 2
Enter burst time of process 3: 8
Enter arrival time of process 4: 3
Enter burst time of process 4: 6

Process Arrival Burst   Waiting TAT
1         0         5         0         5
2         1         3         4         7
3         2         8         6        14
4         3         6        13        19

Average Waiting Time    = 5.75
Average Turnaround Time = 11.25
```

Figure 7.5: Execution of Question 5 — FCFS scheduling with arrival time.

Question 6

Implement the Shortest-Job-First (non-preemptive) scheduling algorithm with arrival time. Among processes available at the current time, the one with the smallest burst time is selected.

PROGRAM / COMMANDS

```
// File: shortest_job_first_with_arival_time.c
#include <stdio.h>
#define max 30

struct Process {
    int pid, arrival, burst, wt, tat, completed;
};

int main() {
    int i, n, completed = 0;
    float awt = 0, atat = 0;
    struct Process proc[max];

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        proc[i].pid = i + 1;
        proc[i].completed = 0;
        printf("Enter arrival time of process %d: ", i + 1);
        scanf("%d", &proc[i].arrival);
        printf("Enter burst time of process %d: ", i + 1);
        scanf("%d", &proc[i].burst);
    }

    int time = 0;
    printf("\nProcess\tArrival\tBurst\tWaiting\tTAT\n");

    while (completed < n) {
        int minBurst = 999, minIdx = -1;
        for (i = 0; i < n; i++)
```

```

        if (proc[i].completed == 0 &&
            proc[i].arrival <= time &&
            proc[i].burst < minBurst) {
            minBurst = proc[i].burst;
            minIdx = i;
        }

        if (minIdx == -1) {
            for (i = 0; i < n; i++)
                if (proc[i].completed == 0 && proc[i].arrival > time) {
                    time = proc[i].arrival;
                    break;
                }
            continue;
        }

        time += proc[minIdx].burst;
        proc[minIdx].tat = time - proc[minIdx].arrival;
        proc[minIdx].wt = proc[minIdx].tat - proc[minIdx].burst;
        proc[minIdx].completed = 1;
        completed++;

        printf("%d\t%d\t%d\t%d\t%d\n",
            proc[minIdx].pid, proc[minIdx].arrival, proc[minIdx].burst,
            proc[minIdx].wt, proc[minIdx].tat);
    }

    for (i = 0; i < n; i++) {
        awt += proc[i].wt;
        atat += proc[i].tat;
    }

    printf("\nAverage Waiting Time    = %.2f\n", awt / n);
    printf("Average Turnaround Time = %.2f\n", atat / n);
    return 0;
}

```

```

hp@Harshit-PC:~/oslab/lab7$ gcc shortest_job_first_with_arival_time.c -o sjf_at
hp@Harshit-PC:~/oslab/lab7$ ./sjf_at

```

TERMINAL SCREENSHOT

```
hp@Harshit-PC:~/oslab/lab7$ ./sjf_at
Enter the number of processes: 4
Enter arrival time of process 1: 0
Enter burst time of process 1: 7
Enter arrival time of process 2: 2
Enter burst time of process 2: 4
Enter arrival time of process 3: 4
Enter burst time of process 3: 1
Enter arrival time of process 4: 5
Enter burst time of process 4: 4

Process Arrival Burst   Waiting TAT
1         0       7       0         7
3         4       1       3         4
2         2       4       6        10
4         5       4       7        11

Average Waiting Time    = 4.00
Average Turnaround Time = 8.00
```

Figure 7.6: Execution of Question 6 — SJF scheduling with arrival time.

Question 7

Implement Priority Scheduling (non-preemptive) with arrival time. Among the processes that have arrived, select the one with the highest priority (lowest priority number).

PROGRAM / COMMANDS

```
// File: priority_scheduling_with_arival_time.c
#include <stdio.h>
#define max 30

struct Process {
    int pid, arrival, burst, priority, wt, tat, completed;
};

int main() {
    int i, n, completed = 0;
    float awt = 0, atat = 0;
    struct Process proc[max];

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        proc[i].pid = i + 1;
        proc[i].completed = 0;
        printf("Enter arrival time of process %d: ", i + 1);
        scanf("%d", &proc[i].arrival);
        printf("Enter burst time of process %d: ", i + 1);
        scanf("%d", &proc[i].burst);
        printf("Enter priority of process %d (lower = higher priority): ", i + 1);
        scanf("%d", &proc[i].priority);
    }

    int time = 0;
    printf("\nProcess\tArrival\tBurst\tPriority\tWaiting\tTAT\n");

    while (completed < n) {
```

```

int minPriority = 999, minIdx = -1;

for (i = 0; i < n; i++)
    if (proc[i].completed == 0 &&
        proc[i].arrival <= time &&
        proc[i].priority < minPriority) {
        minPriority = proc[i].priority;
        minIdx = i;
    }

if (minIdx == -1) {
    for (i = 0; i < n; i++)
        if (proc[i].completed == 0 && proc[i].arrival > time) {
            time = proc[i].arrival;
            break;
        }
    continue;
}

time += proc[minIdx].burst;
proc[minIdx].tat = time - proc[minIdx].arrival;
proc[minIdx].wt = proc[minIdx].tat - proc[minIdx].burst;
proc[minIdx].completed = 1;
completed++;

printf("%d\t%d\t%d\t%d\t\t%d\t\t%d\n",
        proc[minIdx].pid, proc[minIdx].arrival, proc[minIdx].burst,
        proc[minIdx].priority, proc[minIdx].wt, proc[minIdx].tat);
}

for (i = 0; i < n; i++) {
    awt += proc[i].wt;
    atat += proc[i].tat;
}

printf("\nAverage Waiting Time    = %.2f\n", awt / n);
printf("Average Turnaround Time = %.2f\n", atat / n);
return 0;
}

```

```

hp@Harshit-PC:~/oslab/lab7$ gcc priority_scheduling_with_arival_time.c -o pri_at
hp@Harshit-PC:~/oslab/lab7$ ./pri_at

```

TERMINAL SCREENSHOT

```

hp@Harshit-PC:~/oslab/lab7$ ./pri_at
Enter the number of processes: 4
... (input given as above) ...

Process Arrival Burst  Priority    Waiting TAT
1         0         4         2         0         4
2         1         3         1         3         6
3         2         1         3         6         7
4         3         5         4         5        10

Average Waiting Time    = 3.50
Average Turnaround Time = 6.75

```

Figure 7.7: Execution of Question 7 — Priority scheduling with arrival time.

Question 8

Implement Round Robin scheduling with arrival time. Each process is allocated the CPU for a fixed time quantum in a circular order based on its arrival time.

PROGRAM / COMMANDS

```
// File: round_robin_with_arival_time.c
#include <stdio.h>
#define max 30

struct Process {
    int pid, arrival, burst, remaining, completion, wt, tat;
};

int main() {
    int i, n, completed = 0, tq, time = 0;
    float awt = 0, atat = 0;
    struct Process proc[max];

    printf("Enter the number of processes: ");
    scanf("%d", &n);
    printf("Enter time quantum: ");
    scanf("%d", &tq);

    for (i = 0; i < n; i++) {
        proc[i].pid = i + 1;
        printf("Enter arrival time of process %d: ", i + 1);
        scanf("%d", &proc[i].arrival);
        printf("Enter burst time of process %d: ", i + 1);
        scanf("%d", &proc[i].burst);
        proc[i].remaining = proc[i].burst;
        proc[i].completion = 0;
    }

    printf("\nProcess\tArrival\tBurst\tWaiting\tTAT\n");

    while (completed < n) {
        int executed = 0;
        for (i = 0; i < n; i++) {
            if (proc[i].remaining > 0 && proc[i].arrival <= time) {
                executed = 1;
                if (proc[i].remaining <= tq) {
                    time += proc[i].remaining;
                    proc[i].completion = time;
                    proc[i].tat = proc[i].completion - proc[i].arrival;
                    proc[i].wt = proc[i].tat - proc[i].burst;
                    proc[i].remaining = 0;
                    completed++;
                    printf("%d\t%d\t%d\t%d\t%d\n",
                        proc[i].pid, proc[i].arrival, proc[i].burst,
                        proc[i].wt, proc[i].tat);
                } else {
                    time += tq;
                    proc[i].remaining -= tq;
                }
            }
        }

        if (executed == 0) {
            int minArrival = 999;
            for (i = 0; i < n; i++)
                if (proc[i].remaining > 0 && proc[i].arrival > time &&
                    proc[i].arrival < minArrival)
                    minArrival = proc[i].arrival;
            if (minArrival != 999) time = minArrival;
        }
    }
}
```

```

    for (i = 0; i < n; i++) {
        awt += proc[i].wt;
        atat += proc[i].tat;
    }

    printf("\nAverage Waiting Time    = %.2f\n", awt / n);
    printf("Average Turnaround Time = %.2f\n", atat / n);
    return 0;
}

```

```

hp@Harshit-PC:~/oslab/lab7$ gcc round_robin_with_arival_time.c -o rr_at
hp@Harshit-PC:~/oslab/lab7$ ./rr_at

```

TERMINAL SCREENSHOT

```

hp@Harshit-PC:~/oslab/lab7$ ./rr_at
Enter the number of processes: 3
Enter time quantum: 2
... (arrival/burst values entered) ...

Process Arrival Burst   Waiting TAT
3         2         3       4        7
1         0         5       6       11
2         1         4       7       11

Average Waiting Time    = 5.67
Average Turnaround Time = 9.67

```

Figure 7.8: Execution of Question 8 — Round Robin scheduling with arrival time.

Lab 8

Memory Management — Partitioning and Page Replacement Algorithms

Main Objective

The objective of this lab is to implement and analyze **memory management techniques** in operating systems. The first part covers **memory partitioning** (Fixed and Variable) used in contiguous memory allocation. The second part implements major **page replacement algorithms** — FIFO, LRU, LFU, and MFU — used in virtual memory management to decide which page should be evicted from main memory when a page fault occurs.

Question 1

Write a C program that simulates both Fixed Partitioning and Variable Partitioning (using the First-Fit strategy) of memory. The program should allow the user to choose the strategy, accept partition / process sizes, and display the final memory layout along with utilization statistics.

PROGRAM / COMMANDS

```
// File: fixed_variable_partitioning.c
#include <stdio.h>
#define MAX 100

int main() {
    int totalMemory, choice;

    printf("==== Memory Partitioning ==== \n");
    printf("1. Fixed Partitioning \n");
    printf("2. Variable Partitioning \n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    if (choice == 1) {
        printf("\n==== Fixed Partitioning ==== \n");
        printf("Enter total memory size: ");
        scanf("%d", &totalMemory);

        int numPartitions;
        printf("Enter number of partitions: ");
        scanf("%d", &numPartitions);

        int partitionSize[MAX], allocated[MAX], process[MAX];
        for (int i = 0; i < numPartitions; i++) {
            printf("Enter size of partition %d: ", i + 1);
            scanf("%d", &partitionSize[i]);
            allocated[i] = 0; process[i] = -1;
        }

        int numProcesses;
        printf("Enter number of processes: ");
        scanf("%d", &numProcesses);

        printf("\nProcess\tSize\tAllocated to Partition\n");
        printf("----- \n");
        for (int i = 0; i < numProcesses; i++) {
            int size;
            printf("Enter size of process %d: ", i + 1);
            scanf("%d", &size);

            int allocated_to = -1;
            for (int j = 0; j < numPartitions; j++)
                if (partitionSize[j] >= size && !allocated[j]) {
                    allocated[j] = 1;
                }
        }
    }
}
```

```

        process[j] = i + 1;
        allocated_to = j + 1;
        break;
    }
    if (allocated_to != -1)
        printf("%d\t%d\tPartition %d (Size: %d)\n",
            i + 1, size, allocated_to, partitionSize[allocated_to - 1]);
    else
        printf("%d\t%d\tNOT ALLOCATED\n", i + 1, size);
}

printf("\nPartition Status:\n");
printf("Partition\tSize\tAllocated\tProcess\tWasted\n");
printf("-----\n");
for (int i = 0; i < numPartitions; i++) {
    if (allocated[i])
        printf("%d\t%d\tYes\tP%d\t-\n", i + 1, partitionSize[i], process[i]);
    else
        printf("%d\t%d\tNo\t\t-\n", i + 1, partitionSize[i], partitionSize[i]);
}
}
else if (choice == 2) {
    printf("\n==== Variable Partitioning (First-Fit) =====\n");
    printf("Enter total memory size: ");
    scanf("%d", &totalMemory);

    int startAddr[MAX], size[MAX], processId[MAX];
    int numAllocations = 0, usedMemory = 0;
    int numProcesses;
    printf("Enter number of processes: ");
    scanf("%d", &numProcesses);

    printf("\nProcess\tSize\tStart\tEnd\tStatus\n");
    printf("-----\n");
    for (int i = 0; i < numProcesses; i++) {
        int psize;
        printf("Enter size of process %d: ", i + 1);
        scanf("%d", &psize);
        if (usedMemory + psize <= totalMemory) {
            startAddr[numAllocations] = usedMemory;
            size[numAllocations] = psize;
            processId[numAllocations] = i + 1;
            usedMemory += psize;
            printf("%d\t%d\t%d\t%d\tALLOCATED\n",
                i + 1, psize, startAddr[numAllocations],
                startAddr[numAllocations] + psize - 1);
            numAllocations++;
        } else {
            printf("%d\t%d\t\t\t\tNOT ALLOCATED\n", i + 1, psize);
        }
    }

    printf("\nMemory Statistics:\n");
    printf("Total Memory : %d\n", totalMemory);
    printf("Used Memory : %d\n", usedMemory);
    printf("Free Memory : %d\n", totalMemory - usedMemory);
    printf("Utilization : %.2f%%\n", (usedMemory * 100.0) / totalMemory);
}
else printf("Invalid choice!\n");

return 0;
}

```

```

hp@Harshit-PC:~/oslab/lab8$ gcc fixed_variable_partitioning.c -o partition
hp@Harshit-PC:~/oslab/lab8$ ./partition

```

TERMINAL SCREENSHOT

```
hp@Harshit-PC:~/oslab/lab8$ ./partition
===== Memory Partitioning =====
1. Fixed Partitioning
2. Variable Partitioning
Enter your choice: 1
... (inputs as above) ...

Partition Status:
Partition  Size  Allocated  Process  Wasted
1          200    Yes        P1        -
2          300    Yes        P2        -
3          200    Yes        P3        -
4          300    No         -        300
```

Figure 8.1: Execution of Question 1 — Fixed and Variable memory partitioning.

Question 2

Implement the FIFO (First-In-First-Out) page replacement algorithm. The program should accept the number of frames, the reference string, and display the frame status, page hits, page faults, and the page fault rate.

PROGRAM / COMMANDS

```
// File: FIFO.c
#include <stdio.h>
#define MAX 100

int main() {
    int frames, pages, pageFaults = 0, pageHits = 0;
    int pageSequence[MAX], frameArray[MAX];

    printf("Enter number of frames: ");
    scanf("%d", &frames);
    printf("Enter number of pages: ");
    scanf("%d", &pages);
    printf("Enter page sequence: ");
    for (int i = 0; i < pages; i++) scanf("%d", &pageSequence[i]);

    for (int i = 0; i < frames; i++) frameArray[i] = -1;

    printf("\nFIFO Page Replacement:\n");
    printf("Page\tFrames\t\t\tStatus\n");
    printf("-----\n");

    int nextSlot = 0;
    for (int i = 0; i < pages; i++) {
        int page = pageSequence[i], found = 0;

        for (int j = 0; j < frames; j++)
            if (frameArray[j] == page) { found = 1; pageHits++; break; }

        if (!found) {
            pageFaults++;
            frameArray[nextSlot] = page;
            nextSlot = (nextSlot + 1) % frames;
        }

        printf("%d\t", page);
        for (int j = 0; j < frames; j++)
            if (frameArray[j] != -1) printf("%d ", frameArray[j]);
        else printf("- ");
    }
}
```

```

    printf("\t%s\n", found ? "HIT" : "MISS");
}

printf("\nTotal Page Faults: %d\n", pageFaults);
printf("Total Page Hits   : %d\n", pageHits);
printf("Page Fault Rate   : %.2f%%\n", (pageFaults * 100.0) / pages);
return 0;
}

```

```

hp@Harshit-PC:~/oslab/lab8$ gcc FIFO.c -o fifo
hp@Harshit-PC:~/oslab/lab8$ ./fifo

```

TERMINAL SCREENSHOT

```

hp@Harshit-PC:~/oslab/lab8$ ./fifo
Enter number of frames: 3
Enter number of pages: 10
Enter page sequence: 7 0 1 2 0 3 0 4 2 3

FIFO Page Replacement:
Page   Frames           Status
-----
7       7 - -           MISS
0       7 0 -           MISS
1       7 0 1           MISS
2       2 0 1           MISS
0       2 0 1           HIT
3       2 3 1           MISS
0       2 3 0           MISS
4       4 3 0           MISS
2       4 2 0           MISS
3       4 2 3           MISS

Total Page Faults: 9
Page Fault Rate   : 90.00%

```

Figure 8.2: Execution of Question 2 — FIFO page replacement algorithm.

Question 3

Implement the LRU (Least Recently Used) page replacement algorithm. The page that has not been used for the longest period of time should be the victim when a replacement is needed.

PROGRAM / COMMANDS

```

// File: LRU.c
#include <stdio.h>
#define MAX 100

int main() {
    int frames, pages, pageFaults = 0, pageHits = 0;
    int pageSequence[MAX], frameArray[MAX], recentlyUsed[MAX];

    printf("Enter number of frames: "); scanf("%d", &frames);
    printf("Enter number of pages: ");   scanf("%d", &pages);
    printf("Enter page sequence: ");
    for (int i = 0; i < pages; i++) scanf("%d", &pageSequence[i]);

    for (int i = 0; i < frames; i++) { frameArray[i] = -1; recentlyUsed[i] = -1; }
}

```

```

printf("\nLRU Page Replacement:\n");
printf("Page\tFrames\t\t\tStatus\n");
printf("-----\n");

int frameCount = 0;
for (int i = 0; i < pages; i++) {
    int page = pageSequence[i], found = -1;
    for (int j = 0; j < frames; j++)
        if (frameArray[j] == page) {
            found = j; pageHits++; recentlyUsed[j] = i; break;
        }

    if (found == -1) {
        pageFaults++;
        if (frameCount < frames) {
            frameArray[frameCount] = page;
            recentlyUsed[frameCount] = i;
            frameCount++;
        } else {
            int minTime = recentlyUsed[0], minIndex = 0;
            for (int j = 1; j < frames; j++)
                if (recentlyUsed[j] < minTime) {
                    minTime = recentlyUsed[j]; minIndex = j;
                }
            frameArray[minIndex] = page;
            recentlyUsed[minIndex] = i;
        }
    }

    printf("%d\t", page);
    for (int j = 0; j < frames; j++)
        if (frameArray[j] != -1) printf("%d ", frameArray[j]);
        else printf("- ");
    printf("\t%s\n", found != -1 ? "HIT" : "MISS");
}

printf("\nTotal Page Faults: %d\n", pageFaults);
printf("Total Page Hits   : %d\n", pageHits);
printf("Page Fault Rate   : %.2f%%\n", (pageFaults * 100.0) / pages);
return 0;
}

```

```

hp@Harshit-PC:~/oslab/lab8$ gcc LRU.c -o lru
hp@Harshit-PC:~/oslab/lab8$ ./lru

```

TERMINAL SCREENSHOT

```
hp@Harshit-PC:~/oslab/lab8$ ./lru
Enter number of frames: 3
Enter number of pages: 10
Enter page sequence: 7 0 1 2 0 3 0 4 2 3
```

LRU Page Replacement:

Page	Frames	Status
7	7 - -	MISS
0	7 0 -	MISS
1	7 0 1	MISS
2	2 0 1	MISS
0	2 0 1	HIT
3	2 0 3	MISS
0	2 0 3	HIT
4	4 0 3	MISS
2	4 0 2	MISS
3	4 3 2	MISS

Total Page Faults: 8

Page Fault Rate : 80.00%

Figure 8.3: Execution of Question 3 — LRU page replacement algorithm.

Question 4

Implement the LFU (Least Frequently Used) page replacement algorithm. On a page fault when all frames are full, the page that has been referenced the least number of times is evicted.

PROGRAM / COMMANDS

```
// File: LFU.c
#include <stdio.h>
#define MAX 100

int main() {
    int frames, pages, pageFaults = 0, pageHits = 0;
    int pageSequence[MAX], frameArray[MAX], frequency[MAX];

    printf("Enter number of frames:"); scanf("%d", &frames);
    printf("Enter number of pages: "); scanf("%d", &pages);
    printf("Enter page sequence: ");
    for (int i = 0; i < pages; i++) scanf("%d", &pageSequence[i]);

    for (int i = 0; i < frames; i++) { frameArray[i] = -1; frequency[i] = 0; }

    printf("\nLFU Page Replacement:\n");
    printf("Page\tFrames\t\t\tStatus\n");
    printf("-----\n");

    int frameCount = 0;
    for (int i = 0; i < pages; i++) {
        int page = pageSequence[i], found = -1;
        for (int j = 0; j < frames; j++)
            if (frameArray[j] == page) {
                found = j; pageHits++; frequency[j]++; break;
            }

        if (found == -1) {
            pageFaults++;
        }
    }
}
```

```

        if (frameCount < frames) {
            frameArray[frameCount] = page;
            frequency[frameCount] = 1;
            frameCount++;
        } else {
            int minFreq = frequency[0], minIndex = 0;
            for (int j = 1; j < frames; j++)
                if (frequency[j] < minFreq) {
                    minFreq = frequency[j]; minIndex = j;
                }
            frameArray[minIndex] = page;
            frequency[minIndex] = 1;
        }
    }

    printf("%d\t", page);
    for (int j = 0; j < frames; j++)
        if (frameArray[j] != -1) printf("%d ", frameArray[j]);
        else printf("- ");
    printf("\t%s\n", found != -1 ? "HIT" : "MISS");
}

printf("\nTotal Page Faults: %d\n", pageFaults);
printf("Total Page Hits : %d\n", pageHits);
printf("Page Fault Rate : %.2f%%\n", (pageFaults * 100.0) / pages);
return 0;
}

```

```

hp@Harshit-PC:~/oslab/lab8$ gcc LFU.c -o lfu
hp@Harshit-PC:~/oslab/lab8$ ./lfu

```

TERMINAL SCREENSHOT

```

hp@Harshit-PC:~/oslab/lab8$ ./lfu
Enter number of frames: 3
Enter number of pages: 10
Enter page sequence: 2 3 2 1 5 2 4 5 3 2

LFU Page Replacement:
Page    Frames                Status
-----
2       2 - -                MISS
3       2 3 -                MISS
2       2 3 -                HIT
1       2 3 1                MISS
5       2 5 1                MISS
2       2 5 1                HIT
4       2 5 4                MISS
5       2 5 4                HIT
3       2 5 3                MISS
2       2 5 3                HIT

Total Page Faults: 6
Page Fault Rate : 60.00%

```

Figure 8.4: Execution of Question 4 — LFU page replacement algorithm.

Question 5

Implement the MFU (Most Frequently Used) page replacement algorithm. On a page fault when all frames are full, the page with the maximum reference count is evicted (assumption: it has already served its purpose).

PROGRAM / COMMANDS

```
// File: MFU.c
#include <stdio.h>
#define MAX 100

int main() {
    int frames, pages, pageFaults = 0, pageHits = 0;
    int pageSequence[MAX], frameArray[MAX], frequency[MAX];

    printf("Enter number of frames: "); scanf("%d", &frames);
    printf("Enter number of pages: "); scanf("%d", &pages);
    printf("Enter page sequence: ");
    for (int i = 0; i < pages; i++) scanf("%d", &pageSequence[i]);

    for (int i = 0; i < frames; i++) { frameArray[i] = -1; frequency[i] = 0; }

    printf("\nMFU Page Replacement:\n");
    printf("Page\tFrames\t\t\tStatus\n");
    printf("-----\n");

    int frameCount = 0;
    for (int i = 0; i < pages; i++) {
        int page = pageSequence[i], found = -1;
        for (int j = 0; j < frames; j++)
            if (frameArray[j] == page) {
                found = j; pageHits++; frequency[j]++; break;
            }

        if (found == -1) {
            pageFaults++;
            if (frameCount < frames) {
                frameArray[frameCount] = page;
                frequency[frameCount] = 1;
                frameCount++;
            } else {
                int maxFreq = frequency[0], maxIndex = 0;
                for (int j = 1; j < frames; j++)
                    if (frequency[j] > maxFreq) {
                        maxFreq = frequency[j]; maxIndex = j;
                    }
                frameArray[maxIndex] = page;
                frequency[maxIndex] = 1;
            }
        }

        printf("%d\t", page);
        for (int j = 0; j < frames; j++)
            if (frameArray[j] != -1) printf("%d ", frameArray[j]);
            else printf("- ");
        printf("\t%s\n", found != -1 ? "HIT" : "MISS");
    }

    printf("\nTotal Page Faults: %d\n", pageFaults);
    printf("Total Page Hits : %d\n", pageHits);
    printf("Page Fault Rate : %.2f%%\n", (pageFaults * 100.0) / pages);
    return 0;
}
```

```
hp@Harshit-PC:~/oslab/lab8$ gcc MFU.c -o mfu
```

TERMINAL SCREENSHOT

```
hp@Harshit-PC:~/oslab/lab8$ ./mfu
Enter number of frames: 3
Enter number of pages: 10
Enter page sequence: 2 3 2 1 5 2 4 5 3 2

MFU Page Replacement:
Page   Frames           Status
-----
2      2 - -             MISS
3      2 3 -             MISS
2      2 3 -             HIT
1      2 3 1            MISS
5      5 3 1            MISS
2      5 3 2            MISS
4      5 3 4            MISS
5      5 3 4            HIT
3      5 3 4            HIT
2      5 2 4            MISS

Total Page Faults: 7
Page Fault Rate  : 70.00%
```

Figure 8.5: Execution of Question 5 — MFU page replacement algorithm.

Lab 9

Disk Scheduling Algorithms

Main Objective

The objective of this lab is to implement and compare the major **disk scheduling algorithms** used by the operating system to service I/O requests for the magnetic disk. For each algorithm, the program computes the **total head movement (seek count)** to service a given queue of disk-block requests, starting from a given head position. The algorithms implemented are: **FCFS, SSTF, SCAN, C-SCAN, LOOK, and C-LOOK**.

Question 1

Implement the FCFS (First-Come-First-Serve) disk scheduling algorithm. The disk head services the requests in the exact order they arrive in the queue. Compute and display the total seek count and head movement sequence.

PROGRAM / COMMANDS

```
// File: first_come_first_serve_disk_scheduling.c
#include <stdio.h>
#include <stdlib.h>

void fcfs(int head, int *queue, int n) {
    int seek_count = 0;
    int current_head = head;

    printf("Head movement sequence: %d", current_head);
    for (int i = 0; i < n; i++) {
        seek_count += abs(queue[i] - current_head);
        printf(" -> %d", queue[i]);
        current_head = queue[i];
    }
    printf("\nTotal seek count: %d\n", seek_count);
}

int main() {
    int n, head;

    printf("Enter the number of requests: ");
    scanf("%d", &n);

    int *queue = (int *)malloc(n * sizeof(int));
    printf("Enter the requests (disk positions):\n");
    for (int i = 0; i < n; i++) scanf("%d", &queue[i]);

    printf("Enter the current head position: ");
    scanf("%d", &head);

    printf("\n--- FCFS Disk Scheduling ---\n");
    fcfs(head, queue, n);
    free(queue);
    return 0;
}
```

```
hp@Harshit-PC:~/oslab/lab9$ gcc first_come_first_serve_disk_scheduling.c -o fcfs_disk
hp@Harshit-PC:~/oslab/lab9$ ./fcfs_disk
```

TERMINAL SCREENSHOT

```
hp@Harshit-PC:~/oslab/lab9$ ./fcfs_disk
Enter the number of requests: 8
Enter the requests (disk positions):
176 79 34 60 92 11 41 114
Enter the current head position: 50

--- FCFS Disk Scheduling ---
Head movement sequence: 50 -> 176 -> 79 -> 34 -> 60 -> 92 -> 11 -> 41 -> 114
Total seek count: 510
```

Figure 9.1: Execution of Question 1 — FCFS disk scheduling algorithm.

Question 2

Implement the SSTF (Shortest Seek Time First) disk scheduling algorithm. The request closest (in terms of cylinder number) to the current head position is serviced next. Display the head movement sequence and total seek count.

PROGRAM / COMMANDS

```
// File: sstf.c
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>

void sstf(int head, int *queue, int n) {
    int seek_count = 0, current_head = head;
    int *visited = (int *)calloc(n, sizeof(int));

    printf("Head movement sequence: %d", current_head);

    for (int i = 0; i < n; i++) {
        int min_distance = INT_MAX, min_index = -1;
        for (int j = 0; j < n; j++)
            if (!visited[j]) {
                int distance = abs(queue[j] - current_head);
                if (distance < min_distance) {
                    min_distance = distance;
                    min_index = j;
                }
            }
        visited[min_index] = 1;
        seek_count += min_distance;
        current_head = queue[min_index];
        printf(" -> %d", current_head);
    }
    printf("\nTotal seek count: %d\n", seek_count);
    free(visited);
}

int main() {
    int n, head;
    printf("Enter the number of requests: ");
    scanf("%d", &n);
    int *queue = (int *)malloc(n * sizeof(int));
    printf("Enter the requests (disk positions):\n");
    for (int i = 0; i < n; i++) scanf("%d", &queue[i]);
    printf("Enter the current head position: ");
    scanf("%d", &head);

    printf("\n--- SSTF Disk Scheduling ---\n");
}
```

```

sstf(head, queue, n);
free(queue);
return 0;
}

```

```

hp@Harshit-PC:~/oslab/lab9$ gcc sstf.c -o sstf
hp@Harshit-PC:~/oslab/lab9$ ./sstf

```

TERMINAL SCREENSHOT

```

hp@Harshit-PC:~/oslab/lab9$ ./sstf
Enter the number of requests: 8
Enter the requests (disk positions):
176 79 34 60 92 11 41 114
Enter the current head position: 50

--- SSTF Disk Scheduling ---
Head movement sequence: 50 -> 41 -> 34 -> 11 -> 60 -> 79 -> 92 -> 114 -> 176
Total seek count: 226

```

Figure 9.2: Execution of Question 2 — SSTF disk scheduling algorithm.

Question 3

Implement the SCAN (Elevator) disk scheduling algorithm. The head moves towards one end of the disk, services all requests on the way, reaches the end, and then reverses direction. Compute the total head movement.

PROGRAM / COMMANDS

```

// File: scan.c
#include <stdio.h>
#include <stdlib.h>

int compare(const void *a, const void *b) { return (*(int *)a - *(int *)b); }

void scan(int head, int *queue, int n, int disk_size) {
    int seek_count = 0, current_head = head;
    qsort(queue, n, sizeof(int), compare);

    int left[n], right[n];
    int left_count = 0, right_count = 0;

    for (int i = 0; i < n; i++)
        if (queue[i] < head) left[left_count++] = queue[i];
        else right[right_count++] = queue[i];

    printf("Head movement sequence: %d", current_head);

    for (int i = 0; i < right_count; i++) {
        seek_count += abs(right[i] - current_head);
        current_head = right[i];
        printf(" -> %d", current_head);
    }

    seek_count += abs(disk_size - 1 - current_head);
    current_head = disk_size - 1;
    printf(" -> %d", current_head);

    for (int i = left_count - 1; i >= 0; i--) {

```

```

        seek_count += abs(left[i] - current_head);
        current_head = left[i];
        printf(" -> %d", current_head);
    }

    printf("\nTotal seek count: %d\n", seek_count);
}

int main() {
    int n, head, disk_size;
    printf("Enter the number of requests: ");
    scanf("%d", &n);
    int *queue = (int *)malloc(n * sizeof(int));
    printf("Enter the requests (disk positions):\n");
    for (int i = 0; i < n; i++) scanf("%d", &queue[i]);
    printf("Enter the current head position: ");
    scanf("%d", &head);
    printf("Enter the disk size: ");
    scanf("%d", &disk_size);

    printf("\n--- SCAN Disk Scheduling ---\n");
    scan(head, queue, n, disk_size);
    free(queue);
    return 0;
}

```

```

hp@Harshit-PC:~/oslab/lab9$ gcc scan.c -o scan
hp@Harshit-PC:~/oslab/lab9$ ./scan

```

TERMINAL SCREENSHOT

```

hp@Harshit-PC:~/oslab/lab9$ ./scan
Enter the number of requests: 8
Enter the requests (disk positions):
176 79 34 60 92 11 41 114
Enter the current head position: 50
Enter the disk size: 200

--- SCAN Disk Scheduling ---
Head movement sequence: 50 -> 60 -> 79 -> 92 -> 114 -> 176 -> 199 -> 41 -> 34 -> 11
Total seek count: 337

```

Figure 9.3: Execution of Question 3 — SCAN (Elevator) disk scheduling algorithm.

Question 4

Implement the C-SCAN (Circular SCAN) disk scheduling algorithm. The head moves to one end, services requests on the way, jumps back to the start (without servicing), then again moves toward the other end. Provides more uniform wait times than SCAN.

PROGRAM / COMMANDS

```

// File: cscan.c
#include <stdio.h>
#include <stdlib.h>

int compare(const void *a, const void *b) { return (*(int *)a - *(int *)b); }

void cscan(int head, int *queue, int n, int disk_size) {

```

```

int seek_count = 0, current_head = head;
qsort(queue, n, sizeof(int), compare);

int left[n], right[n];
int left_count = 0, right_count = 0;

for (int i = 0; i < n; i++)
    if (queue[i] < head) left[left_count++] = queue[i];
    else right[right_count++] = queue[i];

printf("Head movement sequence: %d", current_head);

for (int i = 0; i < right_count; i++) {
    seek_count += abs(right[i] - current_head);
    current_head = right[i];
    printf(" -> %d", current_head);
}

seek_count += abs(disk_size - 1 - current_head);
current_head = disk_size - 1;
printf(" -> %d", current_head);

seek_count += current_head;
current_head = 0;
printf(" -> %d", current_head);

for (int i = 0; i < left_count; i++) {
    seek_count += abs(left[i] - current_head);
    current_head = left[i];
    printf(" -> %d", current_head);
}

printf("\nTotal seek count: %d\n", seek_count);
}

int main() {
    int n, head, disk_size;
    printf("Enter the number of requests: ");
    scanf("%d", &n);
    int *queue = (int *)malloc(n * sizeof(int));
    printf("Enter the requests (disk positions):\n");
    for (int i = 0; i < n; i++) scanf("%d", &queue[i]);
    printf("Enter the current head position: ");
    scanf("%d", &head);
    printf("Enter the disk size: ");
    scanf("%d", &disk_size);

    printf("\n--- C-SCAN Disk Scheduling ---\n");
    cscan(head, queue, n, disk_size);
    free(queue);
    return 0;
}

```

```

hp@Harshit-PC:~/oslab/lab9$ gcc cscan.c -o cscan
hp@Harshit-PC:~/oslab/lab9$ ./cscan

```

TERMINAL SCREENSHOT

```
hp@Harshit-PC:~/oslab/lab9$ ./cscan
Enter the number of requests: 8
Enter the requests (disk positions):
176 79 34 60 92 11 41 114
Enter the current head position: 50
Enter the disk size: 200

--- C-SCAN Disk Scheduling ---
Head movement sequence: 50 -> 60 -> 79 -> 92 -> 114 -> 176 -> 199 -> 0 -> 11 -> 34 -> 41
Total seek count: 389
```

Figure 9.4: Execution of Question 4 — C-SCAN disk scheduling algorithm.

Question 5

Implement the LOOK disk scheduling algorithm. Similar to SCAN, but the head only goes as far as the last request in each direction instead of reaching the end of the disk.

PROGRAM / COMMANDS

```
// File: look.c
#include <stdio.h>
#include <stdlib.h>

int compare(const void *a, const void *b) { return (*(int *)a - *(int *)b); }

void look(int head, int *queue, int n) {
    int seek_count = 0, current_head = head;
    qsort(queue, n, sizeof(int), compare);

    int left[n], right[n];
    int left_count = 0, right_count = 0;

    for (int i = 0; i < n; i++)
        if (queue[i] < head) left[left_count++] = queue[i];
        else right[right_count++] = queue[i];

    printf("Head movement sequence: %d", current_head);

    for (int i = 0; i < right_count; i++) {
        seek_count += abs(right[i] - current_head);
        current_head = right[i];
        printf(" -> %d", current_head);
    }

    for (int i = left_count - 1; i >= 0; i--) {
        seek_count += abs(left[i] - current_head);
        current_head = left[i];
        printf(" -> %d", current_head);
    }

    printf("\nTotal seek count: %d\n", seek_count);
}

int main() {
    int n, head;
    printf("Enter the number of requests: ");
    scanf("%d", &n);
    int *queue = (int *)malloc(n * sizeof(int));
    printf("Enter the requests (disk positions):\n");
    for (int i = 0; i < n; i++) scanf("%d", &queue[i]);
}
```

```

printf("Enter the current head position: ");
scanf("%d", &head);

printf("\n--- LOOK Disk Scheduling ---\n");
look(head, queue, n);
free(queue);
return 0;
}

```

```

hp@Harshit-PC:~/oslab/lab9$ gcc look.c -o look
hp@Harshit-PC:~/oslab/lab9$ ./look

```

TERMINAL SCREENSHOT

```

hp@Harshit-PC:~/oslab/lab9$ ./look
Enter the number of requests: 8
Enter the requests (disk positions):
176 79 34 60 92 11 41 114
Enter the current head position: 50

--- LOOK Disk Scheduling ---
Head movement sequence: 50 -> 60 -> 79 -> 92 -> 114 -> 176 -> 41 -> 34 -> 11
Total seek count: 291

```

Figure 9.5: Execution of Question 5 — LOOK disk scheduling algorithm.

Question 6

Implement the C-LOOK (Circular LOOK) disk scheduling algorithm. Similar to C-SCAN, but instead of going to the end of the disk, the head only goes as far as the last request in the current direction, then jumps directly to the lowest request and continues.

PROGRAM / COMMANDS

```

// File: clook.c
#include <stdio.h>
#include <stdlib.h>

int compare(const void *a, const void *b) { return (*(int *)a - *(int *)b); }

void clook(int head, int *queue, int n) {
    int seek_count = 0, current_head = head;
    qsort(queue, n, sizeof(int), compare);

    int left[n], right[n];
    int left_count = 0, right_count = 0;

    for (int i = 0; i < n; i++)
        if (queue[i] < head) left[left_count++] = queue[i];
        else right[right_count++] = queue[i];

    printf("Head movement sequence: %d", current_head);

    for (int i = 0; i < right_count; i++) {
        seek_count += abs(right[i] - current_head);
        current_head = right[i];
        printf(" -> %d", current_head);
    }

    if (left_count > 0) {

```

```

        seek_count += abs(left[0] - current_head);
        current_head = left[0];
        printf(" -> %d", current_head);

        for (int i = 1; i < left_count; i++) {
            seek_count += abs(left[i] - current_head);
            current_head = left[i];
            printf(" -> %d", current_head);
        }

    printf("\nTotal seek count: %d\n", seek_count);
}

int main() {
    int n, head;
    printf("Enter the number of requests: ");
    scanf("%d", &n);
    int *queue = (int *)malloc(n * sizeof(int));
    printf("Enter the requests (disk positions):\n");
    for (int i = 0; i < n; i++) scanf("%d", &queue[i]);
    printf("Enter the current head position: ");
    scanf("%d", &head);

    printf("\n--- C-LOOK Disk Scheduling ---\n");
    clook(head, queue, n);
    free(queue);
    return 0;
}

```

```

hp@Harshit-PC:~/oslab/lab9$ gcc clook.c -o clook
hp@Harshit-PC:~/oslab/lab9$ ./clook

```

TERMINAL SCREENSHOT

```

hp@Harshit-PC:~/oslab/lab9$ ./clook
Enter the number of requests: 8
Enter the requests (disk positions):
176 79 34 60 92 11 41 114
Enter the current head position: 50

--- C-LOOK Disk Scheduling ---
Head movement sequence: 50 -> 60 -> 79 -> 92 -> 114 -> 176 -> 11 -> 34 -> 41
Total seek count: 322

```

Figure 9.6: Execution of Question 6 — C-LOOK disk scheduling algorithm.

— *End of Operating Systems Lab Manual* —

Harshit Kamriya | 2024BITE037 | IT-A | 4th Semester

Department of Information Technology, NIT Srinagar
